

Module 14 Structured Notes: C++ Class Implementation for Singly Linked Lists

1. Current Progress Analysis

You have already learned the main operations of a **singly linked list** using procedural C-style code.

Before Module 14, you learned these topics:

Previous Topic	What You Learned
Arrays vs linked lists	Arrays allow fast index access, but linked lists grow and shrink more easily.
Node structure	A node stores <code>data</code> and <code>next</code> .
Pointer movement	<code>p = p->next</code> moves a pointer to the next node.
Traversal	Visit every node from <code>first</code> until <code>NULL</code> or <code>nullptr</code> .
Display	Print each node's data.
Count / sum / max / min	These are traversal-based operations.
Search	Check each node until the key is found or the list ends.
Insertion	Create a new node and connect it using links.
Insert last	Use a <code>last</code> pointer to append efficiently.
Sorted insertion	Insert based on value order, not index.
Deletion	Change links first, then free/delete the removed node.
Reversal	Reverse data or reverse actual node links.
Concatenation and merge	Combine two linked lists.
Loop detection	Use slow and fast pointers to detect a cycle.

Module 14 does not introduce a completely new linked-list algorithm.

Instead, it teaches a better way to **organize linked-list code in C++**.

The main shift is:

```
Before Module 14:  
Data and functions are separate.  
first may be global.
```

Functions receive pointers as arguments.

After Module 14:

Data and functions are placed inside a class.

first becomes private.

Operations belong to a LinkedList object.

2. Big Idea of Module 14

Module 14 teaches how to implement a singly linked list using **C++ classes**.

Until now, your code may have looked like this:

```
Node *first = nullptr;

void Display(Node *p);
int Length(Node *p);
void Insert(Node *p, int index, int x);
int Delete(Node *p, int index);
```

This is called a **procedural style**.

The functions are separate from the data.

In Module 14, we change the style to this:

```
LinkedList list(A, 5);

list.Display();
list.Insert(0, 100);
list.Delete(3);
```

This is called an **object-oriented style**.

The linked list data and the linked list operations are placed inside one object.

3. Why Use a Class?

A linked list has two parts:

1. The actual data structure
2. The operations performed on the data structure

The data structure is made of nodes:

```
10 -> 20 -> 30 -> nullptr
```

The operations include:

```
Display  
Length  
Insert  
Delete  
Destroy
```

A class allows us to keep both parts together.

Instead of writing:

```
Display(first);  
Insert(first, 2, 50);  
Delete(first, 3);
```

We write:

```
list.Display();  
list.Insert(2, 50);  
list.Delete(3);
```

This is cleaner because the list manages itself.

4. Main Benefit: Protecting `first`

In earlier modules, `first` was often global:

```
Node *first = nullptr;
```

That means many functions could directly access and change it.

This can be dangerous.

Example mistake:

```
first = nullptr;
```

If this is done accidentally, the entire list can become unreachable.

In Module 14, `first` becomes a **private member** of the `LinkedList` class:

```
class LinkedList {  
private:  
    Node *first;  
};
```

Now outside code cannot directly modify `first`.

This protects the list.

5. Important C++ Terms

Class

A class is a blueprint for creating objects.

Example:

```
class LinkedList {  
    // data and functions  
};
```

This defines what a linked list object should contain and what it can do.

Object

An object is a real instance of a class.

Example:

```
LinkedList list;
```

Here, `list` is an object of the `LinkedList` class.

Member variable

A member variable is data stored inside a class.

Example:

```
Node *first;
```

This stores the address of the first node.

Member function

A member function is a function inside a class.

Example:

```
void Display();  
int Length();  
void Insert(int index, int x);  
int Delete(int index);
```

These functions operate on the linked list object.

Private

`private` means only the class itself can directly access that part.

Example:

```
private:
    Node *first;
```

Outside code cannot write:

```
list.first = nullptr; // not allowed
```

Public

`public` means outside code can use that part.

Example:

```
public:
    void Display();
```

Outside code can write:

```
list.Display();
```

Constructor

A constructor is a special function that runs automatically when an object is created.

Example:

```
LinkedList list;
```

This calls the constructor.

Constructors are used to initialize the object.

Destructor

A destructor is a special function that runs automatically when an object is destroyed.

Example:

```
~LinkedList();
```

The destructor is used to release heap memory used by the linked list.

6. The Two Classes in Module 14

Module 14 uses two classes:

```
class Node  
class LinkedList
```

They have different jobs.

Class	Job
Node	Represents one node in the list.
LinkedList	Represents the whole linked list and its operations.

7. The Node Class

A node stores two things:

```
data + next
```

Example node:

```
[10 | next]
```

In C++, the node can be written like this:

```
class Node {  
public:  
    int data;
```

```
Node *next;  
};
```

Explanation

```
int data;
```

This stores the value inside the node.

Example:

```
data = 10
```

```
Node *next;
```

This stores the address of the next node.

If there is no next node, then:

```
next = nullptr;
```

8. Why `Node` Members Are Public Here

In a strict object-oriented design, node data may also be private.

But in beginner linked-list implementations, the `Node` class is often written like this:

```
class Node {  
public:  
    int data;  
    Node *next;  
};
```

This makes it easier for the `LinkedList` class to access:


```
p->data  
p->next
```

For learning linked lists, this simple design is acceptable.

9. The `LinkedList` Class

The `LinkedList` class represents the entire linked list.

Basic structure:

```
class LinkedList {  
private:  
    Node *first;  
  
public:  
    LinkedList();  
    LinkedList(int A[], int n);  
    ~LinkedList();  
  
    void Display();  
    int Length();  
    void Insert(int index, int x);  
    int Delete(int index);  
};
```

10. Meaning of `first` Inside the Class

```
Node *first;
```

`first` stores the address of the first node.

Example:

```
first  
|
```

```
v
10 -> 20 -> 30 -> nullptr
```

If the list is empty:

```
first -> nullptr
```

In Module 14, `first` belongs to the object.

Example:

```
LinkedList list1;
LinkedList list2;
```

Then:

```
list1 has its own first
list2 has its own first
```

This is better than using one global `first` for everything.

11. Why Member Functions Do Not Need `first` as a Parameter

Before Module 14, display may have been written like this:

```
void Display(Node *p) {
    while (p != nullptr) {
        cout << p->data << " ";
        p = p->next;
    }
}
```

You had to call it like this:

```
Display(first);
```

In Module 14, `Display()` is inside the `LinkedList` class.

So it can access `first` directly.

Now we write:

```
void Display() {
    Node *p = first;

    while (p != nullptr) {
        cout << p->data << " ";
        p = p->next;
    }
}
```

And call it like this:

```
list.Display();
```

The object already knows its own `first`.

So we do not need to pass `first` as an argument.

12. Default Constructor

The default constructor creates an empty list.

Code:

```
LinkedList() {
    first = nullptr;
}
```

Meaning

When we write:

```
LinkedList list;
```

The list starts empty.

Picture:

```
first -> nullptr
```

No nodes exist yet.

13. Parameterized Constructor

The parameterized constructor creates a linked list from an array.

Code idea:

```
LinkedList(int A[], int n)
```

Example:

```
int A[] = {10, 20, 30};  
LinkedList list(A, 3);
```

This creates:

```
10 -> 20 -> 30 -> nullptr
```

14. Why the First Node Is Created Separately

When the list is empty, there is no existing node to attach to.

So the first node is special.

Code:

```
first = new Node;  
first->data = A[0];  
first->next = nullptr;  
last = first;
```

Picture after creating the first node:

```
first
|
v
10 -> nullptr
^
|
last
```

Both `first` and `last` point to the same node because there is only one node.

15. Creating the Remaining Nodes

After the first node is created, every other node follows the same pattern.

Code:

```
t = new Node;
t->data = A[i];
t->next = nullptr;

last->next = t;
last = t;
```

Meaning

```
t = new Node;
```

Create a new node in heap memory.

```
t->data = A[i];
```

Store the array value in the new node.

```
t->next = nullptr;
```

The new node is currently the last node.

```
last->next = t;
```

Attach the new node after the current last node.

```
last = t;
```

Move `last` to the new final node.

16. Constructor Dry Run

Given:

```
int A[] = {1, 2, 3};  
LinkedList list(A, 3);
```

Step 1: Create the first node

```
first -> 1 -> nullptr  
last  -> 1
```

Step 2: Add 2

Create new node:

```
2 -> nullptr
```

Connect it:

```
last->next = t;  
last = t;
```

Now:

```
first -> 1 -> 2 -> nullptr
          ^
          last
```

Step 3: Add 3

Create new node:

```
3 -> nullptr
```

Connect it:

```
first -> 1 -> 2 -> 3 -> nullptr
          ^
          last
```

Final list:

```
1 -> 2 -> 3 -> nullptr
```

17. Destructor

The destructor deletes all nodes when the linked-list object is destroyed.

Code:

```
~LinkedList() {
    Node *p = first;

    while (first != nullptr) {
        first = first->next;
        delete p;
        p = first;
    }
}
```

18. Why the Destructor Is Needed

In C++, this creates heap memory:

```
new Node;
```

Heap memory does not automatically disappear immediately after we stop using it.

So we must release it using:

```
delete p;
```

If we do not delete the nodes, the program may have a **memory leak**.

A memory leak means memory was allocated but never released.

The destructor prevents this by deleting every node in the list.

19. Destructor Dry Run

Suppose the list is:

```
first -> 10 -> 20 -> 30 -> nullptr
```

Start:

```
p = first;
```

So:

```
p -> 10  
first -> 10
```


Loop 1

```
first = first->next;  
delete p;  
p = first;
```

Result:

```
Deleted node 10  
first -> 20 -> 30 -> nullptr  
p -> 20
```

Loop 2

```
Deleted node 20  
first -> 30 -> nullptr  
p -> 30
```

Loop 3

```
Deleted node 30  
first -> nullptr  
p -> nullptr
```

Now `first == nullptr`, so the loop stops.

All nodes have been deleted.

20. Important Destructor Rule

The safe order is:

```
first = first->next;  
delete p;  
p = first;
```

Why?

Because before deleting the current node, we save the address of the next node by moving `first` forward.

This would be unsafe:

```
delete first;
first = first->next; // wrong
```

After `delete first`, we should not access `first->next`.

The node has already been deleted.

21. Display Function Inside the Class

Code:

```
void Display() {
    Node *p = first;

    while (p != nullptr) {
        cout << p->data << " ";
        p = p->next;
    }

    cout << endl;
}
```

Meaning

```
Node *p = first;
```

Use a temporary pointer `p` to traverse the list.

Do not move `first` directly.

```
while (p != nullptr)
```

Keep moving while `p` points to a real node.

```
cout << p->data << " ";
```

Print the current node's data.

```
p = p->next;
```

Move to the next node.

22. Display Dry Run

List:

```
10 -> 20 -> 30 -> nullptr
```

Start:

```
p -> 10
```

Step	p points to	Action
1	10	Print 10, move to 20
2	20	Print 20, move to 30
3	30	Print 30, move to nullptr
4	nullptr	Stop

Output:

```
10 20 30
```

23. Length Function Inside the Class

Code:

```

int Length() {
    Node *p = first;
    int count = 0;

    while (p != nullptr) {
        count++;
        p = p->next;
    }

    return count;
}

```

Meaning

`Length()` counts how many nodes are in the list.

It uses the same traversal pattern as `display`.

The difference is that instead of printing, it increments a counter.

24. Length Dry Run

List:

10 -> 20 -> 30 -> nullptr

Start:

count = 0
p -> 10

Step	p points to	Action	count
1	10	Count node	1
2	20	Count node	2
3	30	Count node	3
4	nullptr	Stop	3

Return:

25. Insert Function Inside the Class

Code:

```
void Insert(int index, int x) {
    if (index < 0 || index > Length()) {
        return;
    }

    Node *t = new Node;
    t->data = x;
    t->next = nullptr;

    if (index == 0) {
        t->next = first;
        first = t;
    } else {
        Node *p = first;

        for (int i = 0; i < index - 1; i++) {
            p = p->next;
        }

        t->next = p->next;
        p->next = t;
    }
}
```

26. Insert Index Convention

This is the same convention used earlier for singly linked-list insertion.

```
index 0 = insert before the first node
index 1 = insert after the first node
index 2 = insert after the second node
index 3 = insert after the third node
```

Example list:

```
10 -> 20 -> 30 -> nullptr
```

Valid insertion indexes:

```
0, 1, 2, 3
```

Index	Meaning	Result if inserting 99
0	Before 10	99 -> 10 -> 20 -> 30
1	After 10	10 -> 99 -> 20 -> 30
2	After 20	10 -> 20 -> 99 -> 30
3	After 30	10 -> 20 -> 30 -> 99

27. Insert Validation

Code:

```
if (index < 0 || index > Length()) {  
    return;  
}
```

This prevents invalid insertion.

If the list has 3 nodes, valid indexes are:

```
0, 1, 2, 3
```

Invalid indexes include:

```
-1, 4, 5, 100
```

If the index is invalid, the function stops and changes nothing.

28. Insert at Head

If `index == 0`, the new node becomes the first node.

Code:

```
t->next = first;  
first = t;
```

Example:

```
Before:  
first -> 10 -> 20 -> 30 -> nullptr  
  
Insert 5 at index 0  
  
After:  
first -> 5 -> 10 -> 20 -> 30 -> nullptr
```

Why this order matters

Correct order:

```
t->next = first;  
first = t;
```

First, the new node points to the old first node.

Then `first` moves to the new node.

If we changed `first` first, we could lose the old list.

29. Insert in the Middle or End

For non-zero index, we first move `p` to the node after which we want to insert.

Code:

```
Node *p = first;

for (int i = 0; i < index - 1; i++) {
    p = p->next;
}
```

Then we connect the new node:

```
t->next = p->next;
p->next = t;
```

Example:

```
Before:
10 -> 20 -> 30 -> nullptr

Insert 25 at index 2
```

Index 2 means insert after the second node.

The second node is 20.

So:

```
p -> 20
```

Then:

```
t->next = p->next;
p->next = t;
```

Result:

```
10 -> 20 -> 25 -> 30 -> nullptr
```

30. Why Insert Link Order Matters

Correct order:


```
t->next = p->next;  
p->next = t;
```

This means:

1. The new node points to the old next node.
2. The previous node points to the new node.

Wrong order:

```
p->next = t;  
t->next = p->next;
```

Why is this wrong?

After:

```
p->next = t;
```

`p->next` no longer points to the original next node.

It points to `t`.

Then:

```
t->next = p->next;
```

makes `t->next` point to `t` itself.

This can create a self-loop and lose the rest of the list.

Important rule:

```
Copy the old link before overwriting it.
```

31. Delete Function Inside the Class

Code:

```

int Delete(int index) {
    if (index < 1 || index > Length()) {
        return -1;
    }

    Node *p = first;
    Node *q = nullptr;
    int x;

    if (index == 1) {
        first = first->next;
        x = p->data;
        delete p;
        return x;
    }

    for (int i = 0; i < index - 1; i++) {
        q = p;
        p = p->next;
    }

    q->next = p->next;
    x = p->data;
    delete p;

    return x;
}

```

32. Delete Index Convention

Deletion uses 1-based indexing.

```

index 1 = delete the first node
index 2 = delete the second node
index 3 = delete the third node

```

Example list:

```

10 -> 20 -> 30 -> nullptr

```

Delete index	Node deleted	Final list
1	10	20 -> 30
2	20	10 -> 30
3	30	10 -> 20

Important:

Insertion allows index 0.
Deletion does not allow index 0.

For deletion, index 0 is invalid.

33. Delete Validation

Code:

```
if (index < 1 || index > Length()) {
    return -1;
}
```

This means:

Condition	Meaning
<code>index < 1</code>	The index is too small.
<code>index > Length()</code>	The index is larger than the number of nodes.

If the index is invalid, the function returns:

-1

and the list is not changed.

34. Deleting the First Node

Code:

```
if (index == 1) {  
    first = first->next;  
    x = p->data;  
    delete p;  
    return x;  
}
```

Example:

Before:
first -> 10 -> 20 -> 30 -> nullptr

Delete index 1

At the start:

```
p -> 10  
first -> 10
```

Move `first`:

```
first = first->next;
```

Now:

```
first -> 20 -> 30 -> nullptr  
p -> 10
```

Save data:

```
x = p->data;
```

So:

```
x = 10
```

Delete old first node:

```
delete p;
```

Final list:

```
20 -> 30 -> nullptr
```

Return:

```
10
```

35. Deleting a Middle or Last Node

For index greater than 1, we use two pointers:

Pointer	Meaning
<code>p</code>	Points to the node to delete.
<code>q</code>	Points to the node before <code>p</code> .

Example:

```
10 -> 20 -> 30 -> 40 -> nullptr
```

Delete index 3.

At the correct position:

```
10 -> 20 -> 30 -> 40 -> nullptr
      q      p
```

To remove `30`, write:

```
q->next = p->next;
```

This makes 20 point directly to 40.

Then:

```
x = p->data;  
delete p;  
return x;
```

Final list:

```
10 -> 20 -> 40 -> nullptr
```

Returned value:

```
30
```

36. Deleting the Last Node

Deleting the last node uses the same logic as deleting a middle node.

Example:

```
10 -> 20 -> 30 -> nullptr
```

Delete index 3.

At the correct position:

```
10 -> 20 -> 30 -> nullptr  
      q      p
```

Code:

```
q->next = p->next;
```

Since `p->next` is `nullptr`, this becomes:

```
q->next = nullptr;
```

Final list:

```
10 -> 20 -> nullptr
```

Returned value:

```
30
```

37. C Memory vs C++ Memory

This is very important.

In C:

```
malloc  
free
```

In C++:

```
new  
delete
```

Do not mix them.

Wrong:

```
Node *p = new Node;  
free(p); // wrong
```

Correct:

```
Node *p = new Node;  
delete p; // correct
```

Wrong:

```
Node *p = (Node *)malloc(sizeof(Node));  
delete p; // wrong
```

Correct in C style:

```
Node *p = (Node *)malloc(sizeof(Node));  
free(p);
```

Module 14 is C++ style, so use:

```
new
```

and:

```
delete
```

38. NULL vs nullptr

Older C and C++ code often uses:

```
NULL
```

Modern C++ prefers:

```
nullptr
```

Both mean:

```
This pointer points to nothing.
```

In Module 14 C++ code, use:

```
nullptr
```

Example:


```
first = nullptr;
t->next = nullptr;
```

39. Full Working C++ Program

```
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node *next;
};

class LinkedList {
private:
    Node *first;

public:
    LinkedList() {
        first = nullptr;
    }

    LinkedList(int A[], int n) {
        if (n <= 0) {
            first = nullptr;
            return;
        }

        Node *last;
        Node *t;

        first = new Node;
        first->data = A[0];
        first->next = nullptr;
        last = first;

        for (int i = 1; i < n; i++) {
            t = new Node;
            t->data = A[i];
            t->next = nullptr;

            last->next = t;
```

```

        last = t;
    }
}

~LinkedList() {
    Node *p = first;

    while (first != nullptr) {
        first = first->next;
        delete p;
        p = first;
    }
}

void Display() {
    Node *p = first;

    while (p != nullptr) {
        cout << p->data << " ";
        p = p->next;
    }

    cout << endl;
}

int Length() {
    Node *p = first;
    int count = 0;

    while (p != nullptr) {
        count++;
        p = p->next;
    }

    return count;
}

void Insert(int index, int x) {
    if (index < 0 || index > Length()) {
        return;
    }

    Node *t = new Node;
    t->data = x;
    t->next = nullptr;

    if (index == 0) {
        t->next = first;
    }
}

```

```

        first = t;
    } else {
        Node *p = first;

        for (int i = 0; i < index - 1; i++) {
            p = p->next;
        }

        t->next = p->next;
        p->next = t;
    }
}

int Delete(int index) {
    if (index < 1 || index > Length()) {
        return -1;
    }

    Node *p = first;
    Node *q = nullptr;
    int x;

    if (index == 1) {
        first = first->next;
        x = p->data;
        delete p;
        return x;
    }

    for (int i = 0; i < index - 1; i++) {
        q = p;
        p = p->next;
    }

    q->next = p->next;
    x = p->data;
    delete p;

    return x;
}

};

int main() {
    int A[] = {10, 20, 30, 40, 50};

    LinkedList list(A, 5);

    cout << "Original list: ";

```

```

list.Display();

list.Insert(0, 5);
cout << "After inserting 5 at index 0: ";
list.Display();

list.Insert(3, 25);
cout << "After inserting 25 at index 3: ";
list.Display();

int deleted = list.Delete(4);
cout << "Deleted value: " << deleted << endl;

cout << "After deletion: ";
list.Display();

cout << "Length: " << list.Length() << endl;

return 0;
}

```

40. Expected Output

Initial array:

```
{10, 20, 30, 40, 50}
```

The constructor creates:

```
10 -> 20 -> 30 -> 40 -> 50 -> nullptr
```

Then:

```
list.Insert(0, 5);
```

Result:

```
5 -> 10 -> 20 -> 30 -> 40 -> 50 -> nullptr
```

Then:

```
list.Insert(3, 25);
```

Index 3 means insert after the third node.

Current list:

```
5 -> 10 -> 20 -> 30 -> 40 -> 50
```

The third node is **20**.

So result:

```
5 -> 10 -> 20 -> 25 -> 30 -> 40 -> 50
```

Then:

```
int deleted = list.Delete(4);
```

Deletion index 4 means delete the fourth node.

The fourth node is **25**.

Result:

```
5 -> 10 -> 20 -> 30 -> 40 -> 50
```

So output should be similar to:

```
Original list: 10 20 30 40 50
After inserting 5 at index 0: 5 10 20 30 40 50
After inserting 25 at index 3: 5 10 20 25 30 40 50
Deleted value: 25
After deletion: 5 10 20 30 40 50
Length: 6
```

41. Complexity Analysis

Operation	Time Complexity	Extra Space
Default constructor	$O(1)$	$O(1)$
Constructor from array	$O(n)$	$O(n)$ for nodes
Destructor	$O(n)$	$O(1)$
Display	$O(n)$	$O(1)$
Length	$O(n)$	$O(1)$
Insert at head	$O(1)$	$O(1)$
Insert in middle	$O(n)$	$O(1)$
Insert at end	$O(n)$ without <code>last</code> as a member	$O(1)$
Delete first node	$O(1)$	$O(1)$
Delete middle node	$O(n)$	$O(1)$
Delete last node	$O(n)$	$O(1)$

Why constructor from array is $O(n)$

It creates one node for every array element.

If the array has `n` elements, it creates `n` nodes.

So time is:

$O(n)$

It also stores `n` nodes in heap memory.

So node space is:

$O(n)$

Why display is $O(n)$

Display visits every node once.

Why length is $O(n)$

Length counts every node once.

Why insert can be $O(1)$ or $O(n)$

Head insertion is $O(1)$ because no traversal is needed.

Middle or end insertion is $O(n)$ because the function may need to move through the list.

Why delete can be $O(1)$ or $O(n)$

Deleting the first node is $O(1)$.

Deleting another node is $O(n)$ because the function must reach that position.

Why destructor is $O(n)$

The destructor deletes every node one by one.

42. Important Differences: Procedural Style vs Class Style

Procedural Style	Class Style
Uses global or external <code>first</code> .	Keeps <code>first</code> inside the object.
Functions receive <code>Node *p</code> .	Member functions use the object's own <code>first</code> .
Data and functions are separate.	Data and functions are bundled together.
Outside code may directly modify <code>first</code> .	<code>first</code> is private and protected.
Cleanup must be called manually or remembered.	Destructor automatically cleans up nodes.

43. Common Beginner Mistakes

Mistake 1: Making `first` public

Wrong:

```
class LinkedList {  
public:  
    Node *first;  
};
```

This allows outside code to damage the list.

Better:

```
class LinkedList {  
private:  
    Node *first;  
};
```

Mistake 2: Forgetting the destructor

If nodes are created using `new`, they should be deleted using `delete`.

If there is no destructor, the list may leak memory.

Mistake 3: Moving `first` during display

Wrong:

```
while (first != nullptr) {  
    cout << first->data << " ";  
    first = first->next;  
}
```

This destroys access to the list.

Correct:

```
Node *p = first;  
  
while (p != nullptr) {  
    cout << p->data << " ";
```



```
p = p->next;  
}
```

Mistake 4: Mixing new/delete with malloc/free

Wrong:

```
Node *p = new Node;  
free(p);
```

Correct:

```
Node *p = new Node;  
delete p;
```

Mistake 5: Wrong insertion link order

Wrong:

```
p->next = t;  
t->next = p->next;
```

Correct:

```
t->next = p->next;  
p->next = t;
```

Mistake 6: Treating insertion and deletion indexes the same

Insertion:

```
index 0 is valid
```

Deletion:

```
index 0 is invalid
```

Insertion index 0 means insert before the first node.

Deletion index 1 means delete the first node.

Mistake 7: Forgetting that `Length()` costs $O(n)$

This validation:

```
if (index < 0 || index > Length())
```

calls `Length()`, which traverses the list.

That means insertion validation adds an $O(n)$ pass.

For beginner code, this is fine.

For optimized code, we may store list length as a member variable.

44. Cleaner Design Improvement: Store Length as a Member

The beginner implementation calculates length by traversing the list every time.

Alternative design:

```
class LinkedList {  
private:  
    Node *first;  
    int size;  
};
```

Then:

```
Length()
```

could return `size` in $O(1)$.

But this requires carefully updating `size` during every insertion and deletion.

For Module 14, the simpler traversal-based `Length()` is easier to understand.

45. Cleaner Design Improvement: Store a `last` Pointer

The current class stores only:

```
Node *first;
```

Appending at the end may take $O(n)$.

If we also store:

```
Node *last;
```

then inserting at the end can be faster.

Example:

```
last->next = t;  
last = t;
```

This can make append $O(1)$.

But it also makes deletion and edge cases slightly more complex because `last` must stay correct.

For Module 14, using only `first` is simpler.

46. What You Should Be Able to Write After Module 14

After completing this module, you should be able to write these from memory:

```
class Node
```

```
class LinkedList
```

```
LinkedList()
```

```
LinkedList(int A[], int n)
```

```
~LinkedList()
```

```
void Display()
```

```
int Length()
```

```
void Insert(int index, int x)
```

```
int Delete(int index)
```

You should also be able to explain:

- Why first is private.
- Why member functions do not need first as a parameter.
- Why constructor builds the list.
- Why destructor deletes the list.
- Why new must match delete.
- Why insertion and deletion need careful link order.

47. Module 14 Summary

Module 14 converts linked-list code from procedural style to C++ class style.

The most important idea is:

A LinkedList object should manage its own nodes.

Before:

```
Node *first;  
Display(first);  
Insert(first, 2, 50);  
Delete(first, 3);
```

After:

```
LinkedList list;  
list.Display();  
list.Insert(2, 50);  
list.Delete(3);
```

The class contains:

```
private first pointer  
constructors  
destructor  
display  
length  
insert  
delete
```

The main benefits are:

```
Cleaner code  
Better organization  
Protected first pointer  
Automatic cleanup through destructor  
Multiple independent linked-list objects
```

The main rule is:

```
The object owns the list.  
The object manages the list.  
The object deletes the list.
```

That is the core lesson of Module 14.